

Worklogger: A Complete Tutorial for Non-Rust Developers

Contents

Worklogger: A Complete Tutorial for Non-Rust Developers	2
1. What you are building	2
2. Mental model: hexagonal architecture	4
3. Rust toolchain and Cargo	6
4. Rust basics through Worklogger code	8
5. Workspace layout and crates	13
6. The <code>common</code> crate	14
7. The <code>core</code> domain crate	16
8. The <code>use_cases</code> application layer	18
9. The <code>infrastructure</code> crate (PostgreSQL + <code>sqlx</code>)	21
10. The <code>api</code> crate (Axum HTTP server)	23
11. Authentication: device tokens and JWT	24
12. The <code>sdk</code> HTTP client	25
13. The <code>tui</code> terminal UI (Ratatui)	27
14. The <code>admin</code> CLI (Clap)	29
15. End-to-end data flows	31
16. Framework and crate catalog	32
17. Rebuild blueprint: duplicate the project	33
18. Exercises and next steps	36
19. Glossary	37
Appendix A — Environment variables cheat sheet	38
Appendix B — API surface summary	38
Appendix C — Rust syntax field guide (cheat sheet)	38
Appendix D — How to study the real repo in order	39
Closing	40
20. Deep dive: implementing <code>create-worklog</code> across every layer	40
21. Deep dive: filtering, search DSL, and SQL	43
22. Deep dive: ownership worked examples from this repo	44
23. Deep dive: error mapping across layers	45
24. Testing strategy for a workspace like this	47
25. Operational notes: running, migrating, shipping	47
26. Design decisions explained (why the code looks this way)	48
27. Annotated “day in the life” of a contributor	49
28. Common mistakes when non-Rust developers clone this design ..	50

29. Extended recreate script (checklist form)	50
30. Final reference: crate-by-crate public surface	51
31. Side-by-side: the same idea in TypeScript vs Rust	52
32. Walkthrough: reading <code>filter_worklogs</code> without fear	54
33. Performance and simplicity notes (practical)	54
34. Accessibility of the TUI design for keyboard users	54
35. Copy-paste starter: smallest hexagonal Rust service	55
36. Frequently asked questions	55
37. Study plan: 10 evenings to literacy	56
38. Document map (where things live)	57

Worklogger: A Complete Tutorial for Non-Rust Developers

How to understand — and recreate — a real Rust workspace with hexagonal architecture, PostgreSQL, Axum, and a Ratatui TUI

Meta	
Audience	Developers who know another language (Python, TypeScript, Java, Go, ...) but not Rust
Goal	After reading, you should be able to rebuild a project with the same shape as Worklogger
Estimated length	~40+ pages (print / PDF equivalent)
Project	Worklogger — terminal work-log tracker with HTTP API

1. What you are building

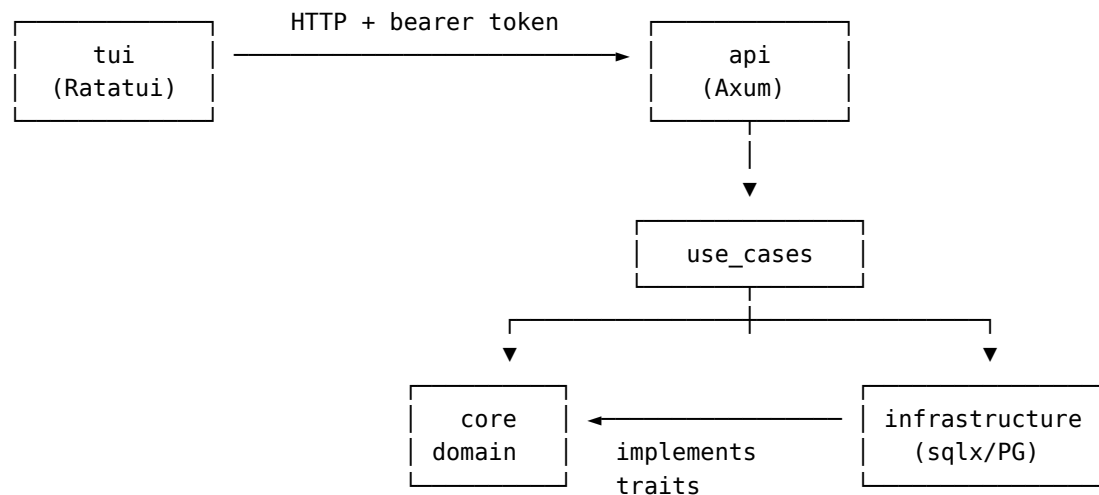
Worklogger is a **work-time logging** system. A user records sessions with:

- a **date** (stored as UTC, shown in the **Jalali** calendar for Asia/Tehran),

- a **duration** (e.g. 2h30m, or raw seconds),
- a **description**,
- one or more **tags**.

They can browse entries in a **keyboard-driven terminal UI**, search with a small DSL (`tag:rust date:1403/01/01..`), soft-delete rows, and **export to Excel**. The same business rules are also exposed as a **REST API** backed by **PostgreSQL**.

1.1 Two front ends, one brain



- **tui** never talks to PostgreSQL. It uses the **sdk** crate (HTTP client) to call **api**.
- **api** loads environment, opens a DB pool, wires repositories into use cases, and serves HTTP.
- **admin** is a separate CLI that talks to the DB (via infrastructure + use cases) to create users and mint device tokens — not meant for end-user laptops in production.

1.2 Why this project is a good Rust teacher

Concept	Where you see it
Ownership & borrowing	Passing <code>&Worklog</code> into <code>repository.save</code>
Result / Option	Almost every public function
Newtype pattern	<code>Email(String)</code> , <code>WorklogDuration(TimeDelta)</code>
Traits as ports	<code>WorklogRepository</code> in <code>core</code> , implemented in <code>infrastructure</code>
Generics	<code>CreateWorklogUseCase<R: WorklogRepository></code>
Async/await	<code>sqlx</code> queries, Axum handlers, SDK calls
Enums + pattern matching	<code>DomainError</code> , <code>TUI Msg / Mode</code>
Workspaces	Root <code>Cargo.toml</code> with 8 members
Derive macros	<code>Serialize</code> , <code>Error</code> , <code>Clap Parser</code>

If you can rebuild this shape in another stack, you already understand the architecture. Learning Rust here is about mapping those ideas onto Rust's type system and tooling.

2. Mental model: hexagonal architecture

Worklogger follows a **ports and adapters** (hexagonal) / clean-architecture style.

2.1 Layers and dependency rule

Inner layers must not import outer layers.

Layer	Crate	Knows about
Domain	<code>core</code>	Itself + <code>common</code> (shared filter/pagination types)
Application	<code>use_cases</code>	<code>core</code> , <code>common</code> , <code>crypto/JWT/xlsx</code> libs — not <code>sqlx</code> , <code>axum</code> , <code>ratatui</code>
Adapters (out)	<code>infrastructure</code>	<code>core</code> traits + <code>sqlx</code>
Adapters (in)	<code>api</code> , <code>tui</code> , <code>admin</code> , <code>sdk</code>	<code>use cases</code> / HTTP / terminal

That means:

- You can swap PostgreSQL for another DB by writing a new adapter that implements the same traits.
- You can add a web UI later without changing `core`.

- Business rules (e.g. “duration must be < 24h”) live in domain/value objects and use-case validation — not in SQL controllers.

2.2 Ports vs adapters (vocabulary)

- **Port:** a Rust **trait** in core, e.g. `WorklogRepository`.
- **Adapter:** a concrete type that implements the port, e.g. `PostgresWorklogRepository`.
- **Use case:** an application service that takes a command DTO, validates, calls domain + ports, returns a response DTO.

Example (simplified from the real code):

```
// PORT (core)
pub trait WorklogRepository {
    async fn save(&self, worklog: &Worklog) -> RepositoryResult<()>;
}

// USE CASE (use_cases) – generic over any R that implements the port
pub struct CreateWorklogUseCase<R> {
    repository: R,
}

impl<R: WorklogRepository> CreateWorklogUseCase<R> {
    pub async fn execute(&self, command: CreateWorklogCommand) ->
    UseCaseResult<CreateWorklogResponse> {
        command.validate()?;
        let worklog = command_to_worklog(command)?;
        self.repository.save(&worklog).await?;
        Ok(CreateWorklogResponse::new(worklog.id()))
    }
}

// ADAPTER (infrastructure)
impl WorklogRepository for PostgresWorklogRepository {
    async fn save(&self, worklog: &Worklog) -> RepositoryResult<()> {
        // sqlx INSERT ...
        Ok(())
    }
}
```

Coming from TypeScript: think “interface in domain package, class in infrastructure package, use-case class that depends on the interface.”

Coming from Java Spring: think “repository interface in domain, JPA impl in infra, @Service use cases.”

3. Rust toolchain and Cargo

3.1 Install Rust

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustc --version    # need a recent stable; Worklogger uses edition 2024
cargo --version
```

Rust's package manager and build tool is **Cargo**. One project (or workspace) has a `Cargo.toml` that lists dependencies and binary/library targets.

3.2 Essential Cargo commands

Command	Meaning
<code>cargo new hello</code>	Create a binary crate
<code>cargo new --lib utils</code>	Create a library crate
<code>cargo build</code>	Compile (debug)
<code>cargo build --release</code>	Optimized binary
<code>cargo run -p api</code>	Run the <code>api</code> package in a workspace
<code>cargo test</code>	Run tests
<code>cargo check</code>	Type-check without full codegen (fast feedback)
<code>cargo add serde --features derive</code>	Add a dependency (needs <code>cargo-edit</code> or edit TOML)

3.3 Workspace (this repo)

Root `Cargo.toml`:

```
[workspace]
members = [
    "core",
    "common",
    "use_cases",
    "infrastructure",
    "tui",
    "api",
    "sdk",
    "admin"
]
resolver = "2"
```

A **workspace** shares one `Cargo.lock` and one target/ build directory. Crates depend on each other with path dependencies:

```
# api/Cargo.toml
domain = { path = "../core", package = "core" }
use_cases = { path = "../use_cases" }
```

Note the rename: the folder/package is named `core`, but dependents often import it as `domain` for readability (use `domain::entities::Worklog`).

3.4 Crates vs packages vs modules (confusing vocabulary)

Term	Meaning
Package	A unit with a <code>Cargo.toml</code> (e.g. <code>api</code>)
Crate	A compilation unit: either a library (<code>lib.rs</code>) or a binary (<code>main.rs</code>)
Module	A namespace inside a crate (<code>mod routes;</code>)
Workspace	A set of packages built together

In Worklogger:

- `core` is a **library** crate (`core/src/lib.rs`).
- `api` is a **binary** crate (`api/src/main.rs`).
- `admin` is a binary with an explicit name `worklogger-admin`.

3.5 Running Worklogger locally (quick path)

Prerequisites: Rust 1.85+, PostgreSQL 14+.

```
# 1. Database
createdb worklog
export DATABASE_URL=postgres://postgres:postgres@127.0.0.1:5432/worklog
cargo install sqlx-cli --no-default-features --features postgres
sqlx migrate run --source infrastructure/migrations

# 2. Env
cp .env.example .env
# edit JWT_SECRET, DATABASE_URL, etc.

# 3. Admin: create user + device token
export WORKLOGGER_ADMIN_TOKEN=secret
cargo run -p admin -- create-user --name Alice --email alice@team.local --
password secret
cargo run -p admin -- create-token --user <uuid> --label laptop
# copy printed wl_... token

# 4. API
export JWT_SECRET='change-me-use-at-least-32-characters-long'
cargo run -p api

# 5. TUI (other terminal)
export WORKLOGGER_BASE_URL=http://127.0.0.1:3000
export WORKLOGGER_TOKEN=wl_...
cargo run -p tui
```

Offline builds without a live DB (sqlx query cache):

```
SQLX_OFFLINE=true cargo build
```

4. Rust basics through Worklogger code

This chapter teaches Rust **only through patterns that appear in this repository**. Skim if you already know Rust; otherwise treat it as the language primer.

4.1 Variables, mutability, and types

```
let host = std::env::var("HOST").unwrap_or_else(|_| "127.0.0.1".to_string());
let mut terminal = setup_terminal()?;
```

- let bindings are **immutable by default**. Use `mut` when you need to change them.
- Types are often inferred; when not, you annotate: `let port: u16 = ....`
- `.to_string()` / `String` = owned, growable UTF-8 string. `&str` = borrowed string slice.

Analogy: `String` $\approx$ Python `str` you own; `&str` $\approx$ a view into someone else's string (or a literal).

4.2 Ownership and borrowing (the big idea)

Rust has one rule that replaces garbage collection for memory safety:

1. Each value has one **owner**.
2. When the owner goes out of scope, the value is dropped (freed).
3. You can either **move** ownership, or **borrow** temporarily with `&` (shared) or `&mut` (exclusive).

In Worklogger:

```
async fn save(&self, worklog: &Worklog) -> RepositoryResult<()>
```

The repository **borrow**s the worklog (`&Worklog`) — it does not take ownership. The caller still owns the entity after save.

```
pub fn tags(&self) -> &Tags {
    &self.tags
}
```

Getter returns a **reference** to internal data so callers cannot accidentally invalidate the entity's invariants by replacing the field freely (fields are private).

```
pub fn soft_delete(&mut self) -> DomainResult<()>
```

`&mut self` means "I need exclusive access to mutate this instance."

Coming from GC languages: you do not free memory yourself; the compiler checks that references never outlive the data.

4.3 Structs and encapsulation

```
pub struct Worklog {
    id: WorklogId,           // private fields
    user_id: UserId,
```

```

    // ...
}

impl Worklog {
    pub fn create(...) -> Self { /* factory */ }
    pub fn reconstitute(...) -> Self { /* load from DB */ }
    pub fn id(&self) -> WorklogId { self.id }
    pub fn soft_delete(&mut self) -> DomainResult<()> { ... }
}

```

- struct = product type (like a class without inheritance).
- impl blocks hold methods.
- Private fields + public constructors is how Worklogger encodes **invariants** (e.g. soft-delete once).

Two construction paths (DDD style):

Method	When
create	Brand-new aggregate (generates UUID, sets timestamps)
reconstitute	Rebuild from persistence without re-running “create” side effects

4.4 Enums and pattern matching

Rust enums are **tagged unions** — each variant can carry data.

```

pub enum DomainError {
    EmptyTag,
    TagTooLong { max: usize, len: usize },
    AlreadyDeleted,
    // ...
}

match app.mode {
    Mode::AddModal => add::view(frame, app),
    Mode::EditModal => edit::view(frame, app),
    Mode::DeleteModal => delete::view(frame, app),
    Mode::OpenModal => open::view(frame, app),
    Mode::Normal | Mode::Search => {}
}

```

matches! macro for boolean checks:

```

pub fn is_modal(self) -> bool {
    matches!(self, Mode::AddModal | Mode::EditModal | Mode::DeleteModal |
    Mode::OpenModal)
}

```

Analogy: TypeScript discriminated unions / Kotlin sealed classes.

4.5 Option and Result — no null, explicit errors

```
pub type DomainResult<T> = Result<T, DomainError>;
```

Type	Meaning
Option<T>	Some(value) or None (nullable)
Result<T, E>	Ok(value) or Err(error)

Common operators (you will see them everywhere):

Syntax	Meaning
<code>?</code>	If Err/None (in Option context with <code>?</code> on Option), return early; else unwrap Ok
<code>.unwrap()</code>	Panic on failure (OK in tests; avoid in libraries)
<code>.expect("msg")</code>	Panic with message (startup config often uses this)
<code>.ok()</code>	Result → Option
<code>.map_err(...)</code>	Transform the error type
<code>if let Some(x) = ...</code>	Conditional unwrap
<code>let Some(x) = ... else { return Err(...) }</code>	Let-else (modern Rust)

From `Worklog::restore`:

```
let Some(_) = self.deleted_at.take() else {  
    return Err(DomainError::NotDeleted);  
};
```

4.6 Traits (interfaces) and generics

```
pub trait WorklogRepository {  
    async fn get(&self, user_id: UserId, id: WorklogId) ->  
RepositoryResult<Worklog>;  
    async fn save(&self, worklog: &Worklog) -> RepositoryResult<>;  
    // ...  
}  
  
impl<R: WorklogRepository> CreateWorklogUseCase<R> {  
    pub async fn execute(&self, command: CreateWorklogCommand) ->  
UseCaseResult<...> { ... }  
}
```

- `R: WorklogRepository` is a **trait bound**: “any type `R` that implements this trait.”
- `Worklogger` also implements the trait for `Arc<R>` so use cases can share pools via atomic reference counting (thread-safe shared ownership).

Coming from Go: traits $\approx$ interfaces, but checked at compile time with monomorphization (or dyn Trait for runtime polymorphism — Worklogger prefers generics + Arc).

4.7 Modules and visibility

```
// core/src/lib.rs
pub mod actor;
pub mod entities;
pub mod traits;
pub mod value_objects;

// core/src/value_objects/mod.rs
mod email;           // private module
pub use email::Email; // re-export public type
```

Keyword	Visibility
(none)	Private to parent module
pub	Public outside the crate
pub(crate)	Visible anywhere inside this crate only
mod foo;	Load foo.rs or foo/mod.rs

API crate example:

```
// api/src/main.rs
mod dto;
mod routes;
mod state;
```

Binary crates use `mod` to split files; library crates expose a public API through `pub use` and `pub mod`.

4.8 Error handling with `thiserror`

```
use thiserror::Error;

#[derive(Debug, Error, PartialEq, Eq)]
pub enum UseCaseError {
    #[error(transparent)]
    Validation(#[from] ValidationError),
    #[error(transparent)]
    Domain(#[from] DomainError),
    #[error(transparent)]
    Repository(#[from] RepositoryError),
    #[error(transparent)]
    Auth(#[from] AuthError),
    #[error("export failed: {0}")]
    Export(String),
}
```

- `#[from]` implements `From<Inner>` for `UseCaseError`, which makes `?` convert automatically.

- `#[error("...")]` implements `Display`.
- Errors are **typed**, not stringly — callers can match on variants.

Layering of errors:

```
DomainError → UseCaseError → ApiError / SdkError
RepositoryError ↗
AuthError —————→
```

4.9 Async Rust (Tokio)

Rust async uses **futures** polled by a runtime. Worklogger uses **Tokio**.

```
#[tokio::main]
async fn main() {
    let pool = connect(&database_url).await.expect("...");
    axum::serve(listener, app).await.expect("server error");
}
```

The TUI is mostly sync (terminal drawing) but needs async for HTTP, so it builds a runtime manually:

```
fn main() -> io::Result<()> {
    let rt = tokio::runtime::Runtime::new()?;
    let handle = rt.handle().clone();
    let app = rt.block_on(wiringup())?;
    run_terminal(&mut terminal, &handle, app)
}
```

Mental model: `async fn` returns a future; `.await` yields until it completes. Do not block inside async with heavy sync work without care.

4.10 Derive macros and Serde

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Worklog { ... }

#[derive(Debug, Clone, Deserialize, Serialize)]
pub struct PagingParams {
    pub page: u32,
    pub size: u32,
}
```

- `Debug` = printable with `{:?}`.
- `Clone` = deep/shallow copy via `.clone()`.
- `Serialize/Deserialize (Serde)` = JSON (and other formats) mapping for API DTOs.

4.11 Newtypes (value objects)

Instead of passing raw `String` for emails:

```
pub struct Email(String);

impl Email {
    pub fn try_new(value: impl Into<String>) -> DomainResult<Self> {
        let trimmed = value.into().trim().to_owned();
    }
}
```

```

        if trimmed.is_empty() { return Err(DomainError::EmptyEmail); }
        if !trimmed.contains('@') { return Err(DomainError::InvalidEmail); }
        Ok(Self(trimmed))
    }
}

```

Benefits:

- Impossible to accidentally pass a username where an email is required (different types).
- Validation happens once at the boundary.
- Same pattern for WorklogDuration, UserId, Tags, etc.

4.12 Collections and iterators

```

let tags: Vec<Tag> = values
    .into_iter()
    .map(|value| Tag::try_from(value.as_ref()))
    .collect::<DomainResult<Vec<_>>>()?;

```

- Vec<T> = growable array.
- Iterators are lazy pipelines (.map, .filter, .collect).
- Collecting Results: if any item is Err, the whole collect is Err.

4.13 Smart pointers you will see: Arc

```

let worklog_repo = Arc::new(PostgresWorklogRepository::new(pool.clone()));

```

Arc<T> = **Atomic Reference Counted** shared ownership. Axum state is Clone and shared across tasks; cloning Arc increments a counter, it does not deep-copy the pool.

5. Workspace layout and crates

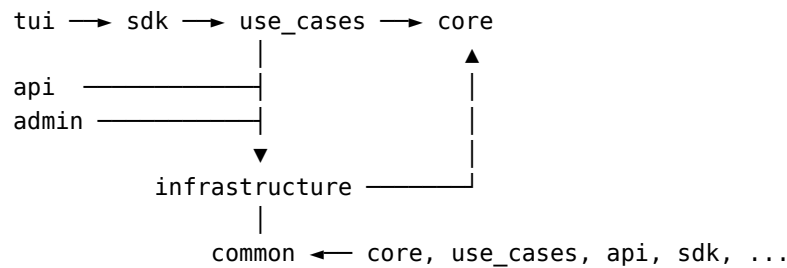
5.1 Directory tree (conceptual)

```

worklogger/
├─ Cargo.toml           # workspace members
├─ Cargo.lock
├─ README.md
├─ .env.example
├─ docs/TUTORIAL.md     # this file
├─ common/              # shared filters & pagination
├─ core/                # domain entities, VOs, repository traits
├─ use_cases/           # application services, auth, xlsx export
├─ infrastructure/      # PostgreSQL adapters + SQL migrations
│   └─ migrations/
├─ api/                 # Axum HTTP server binary
├─ sdk/                 # HTTP client library (used by TUI)
├─ tui/                 # Ratatui terminal UI binary
└─ admin/               # Clap admin CLI binary

```

5.2 Dependency graph (allowed edges)



Forbidden: core depending on infrastructure or api. That would invert the architecture.

5.3 What each crate owns

Crate	Kind	Responsibility
common	lib	PagingParams, PageResult, filter DTOs (ListFilter, JalaliDateFilter, ...)
core	lib	Worklog, User, ApiToken, value objects, *Repository traits, DomainError
use_cases	lib	Commands/responses, validation, use-case structs, JWT/password helpers, Excel export
infrastructure	lib	Postgres*Repository, connection pool, migrations
api	bin	HTTP routes, middleware, JSON DTOs, AppState wiring
sdk	lib	WorkloggerClient over request
tui	bin	Elm-style UI loop, search DSL, dialogs
admin	bin	User/token management CLI

6. The common crate

Path: common/

Role: Tiny shared library so core, use_cases, and HTTP layers agree on filter and pagination shapes without pulling domain entities into every DTO.

6.1 Modules

```

// common/src/lib.rs
pub mod pagination;
pub mod filter;
pub mod util;

```

6.2 Pagination

```
pub struct PagingParams {
    pub page: u32,
    pub size: u32,
}

impl PagingParams {
    pub fn offset(&self) -> u32 {
        (self.page.saturating_sub(1)) * self.size
    }
}

pub struct PageResult<T> {
    pub items: Vec<T>,
    pub total_items: u64,
    pub total_pages: u32,
    pub current_page: u32,
    pub page_size: u32,
}
```

Rust lesson: **saturating_sub** avoids underflow panic (`0u32 - 1`). Generic `PageResult<T>` is like `Page<T>` in Java Spring Data.

Default page size is 20; the TUI often requests 30.

6.3 Filters

Exported types include:

- `ListFilter` — include/exclude lists (tags, IDs)
- `TextFilter` — contains / equals style text matching
- `JalaliDateFilter` / `DateFilter` — date ranges
- `DurationFilter` — duration ranges

These are **transport-friendly** structures. Use cases map them into domain `WorklogFilterCriteria` before hitting the repository.

6.4 Dependencies

Crate	Why
<code>serde</code>	Serialize filters for JSON API
<code>chrono</code> / <code>chrono-tz</code>	Date handling
<code>jalali-rs</code>	Jalali calendar helpers
<code>regex</code>	Parsing helpers in util

When rebuilding: keep common dependency-light. Do not put entities here.

7. The core domain crate

Path: core/

Package name: core (imported as domain elsewhere)

Dependencies: chrono, uuid, thiserror, serde, common

This is the **heart** of the system. No HTTP, no SQL, no UI.

7.1 Module map

Module	Contents
entities	Worklog, User, ApiToken, refresh token entity
value_objects	IDs, Email, Tags, WorklogDuration, timestamps, ...
traits	Repository ports + RepositoryError
criteria	WorklogFilterCriteria for queries
results	WorklogFilterResult (page + stats)
error	DomainError
actor	ActorContext (authenticated user id)
bootstrap	Legacy bootstrap user id constant

7.2 Entity: Worklog

Behavior lives on the entity:

- create / reconstitute
- getters
- soft_delete / restore
- setters that call private touch() to bump updated_at

Soft delete:

```
pub fn soft_delete(&mut self) -> DomainResult<()> {
    if self.is_deleted() {
        return Err(DomainError::AlreadyDeleted);
    }
    self.deleted_at = Some(DeletedAt::new(Utc::now()));
    self.touch();
    Ok(())
}
```

Rust lesson: returning Result<(), DomainError> instead of throwing exceptions. Callers use ? or match.

7.3 Entity: User and ApiToken

User supports disable/enable/soft-delete and password hash storage (hash only — hashing algorithms live in use_cases).

ApiToken stores a **hash** of the device token (Vec<u8>), never the plaintext after minting. Validity:

```
pub fn is_valid(&self) -> bool {
    !self.is_revoked() && !self.is_expired()
}
```

7.4 Value object: WorklogDuration

Rules encoded in the type:

- duration > 0
- duration < 24 hours (86_400 seconds)

```
pub fn try_from_secs(secs: u64) -> DomainResult<Self> {
    if secs == 0 {
        return Err(DomainError::InvalidDuration);
    }
    Self::try_new(TimeDelta::seconds(secs as i64))
}
```

Also implements TryFrom<TimeDelta> and Display. Prefer try_* constructors that return Result over panicking constructors for domain input.

7.5 Value object: Tags / Tag

- At least one tag required (for worklogs)
- Max 50 tags
- Each tag has max length (TAG_MAX_LEN)

```
pub fn try_from_strs(values: impl IntoIterator<Item = impl AsRef<str>>) ->
DomainResult<Self>
```

Rust lesson: impl IntoIterator<Item = impl AsRef<str>> accepts Vec<String>, &[&str], etc. — flexible without forcing callers into one collection type.

7.6 Repository trait (port)

```
pub trait WorklogRepository {
    async fn get(&self, user_id: UserId, id: WorklogId) ->
RepositoryResult<Worklog>;
    async fn save(&self, worklog: &Worklog) -> RepositoryResult<()>;
    async fn update(&self, worklog: &Worklog) -> RepositoryResult<()>;
    async fn filter(&self, criteria: &WorklogFilterCriteria) ->
RepositoryResult<WorklogFilterResult>;
    async fn delete(&self, user_id: UserId, id: WorklogId) ->
RepositoryResult<()>;
}
```

All operations are **scoped by user_id** so one user cannot read another's logs (authorization at persistence boundary as well as API).

Similar traits exist for users, API tokens, and refresh tokens.

7.7 ActorContext

```
pub struct ActorContext {
    user_id: UserId,
}
```

Filled by API auth middleware and passed into controllers via Axum Extension. Commands then get the real `user_id` from the actor, not from untrusted JSON body.

7.8 How to recreate core from scratch

1. `cargo new --lib core`
2. Add dependencies: `chrono`, `uuid (v4)`, `thiserror`, `serde`
3. Define `DomainError` enum
4. Create newtypes for IDs and fields with validation
5. Create entities with private fields + `create/reconstitute`
6. Define repository traits with `async fn`
7. Add unit tests under `#[cfg(test)]` modules (see `user.rs` tests)

Do **not** add `sqlx` here.

8. The `use_cases` application layer

Path: `use_cases/`

Role: Orchestrate one user intention end-to-end: validate input → map to domain → call ports → map to response.

8.1 Layout

```
use_cases/src/
├─ lib.rs
├─ error.rs           # UseCaseError, ValidationError, AuthError
├─ jalali.rs          # parse Jalali dates → UTC datetime
├─ auth/              # password hash (argon2), JWT issue/validate, token
hashing
├─ dtos/
│   └─ commands/      # CreateWorklogCommand, LoginCommand, ...
│   └─ responses/     # CreateWorklogResponse, AuthTokensResponse, ...
├─ mappers/           # command → entity helpers
├─ use_cases/         # one file per use case
└─ export/            # xlsx generation (rust_xlsxwriter)
```

8.2 Command / response DTOs

Commands are **plain data** entering the application layer:

```
pub struct CreateWorklogCommand {
    pub user_id: UserId,
    pub jalali_date: Option<String>,
    pub duration_secs: u64,
    pub tags: Vec<String>,
    pub description: String,
}
```

Validation lives on the command:

```
impl CreateWorklogCommand {
    pub fn validate(&self) -> UseCaseResult<()> {
        if self.duration_secs == 0 { return
Err(ValidationError::DurationRequired.into()); }
        if self.duration_secs >= 86_400 { return
Err(ValidationError::DurationTooLong.into()); }
        if self.tags.is_empty() { return
Err(ValidationError::TagsRequired.into()); }
        // ...
        Ok(())
    }
}
```

Why validate twice (command + domain VO)?

Command validation gives friendly application errors early (API can return 400 with messages). Domain VOs still protect invariants if something calls `WorklogDuration::try_from_secs` directly.

8.3 Use case pattern (template)

Every use case follows roughly:

```
pub struct FooUseCase<R> {
    repository: R,
}

impl<R> FooUseCase<R> {
    pub fn new(repository: R) -> Self { Self { repository } }
}

impl<R: SomeRepository> FooUseCase<R> {
    pub async fn execute(&self, command: FooCommand) ->
UseCaseResult<FooResponse> {
        command.validate()?;
        // domain work + repository calls
        Ok(FooResponse { ... })
    }
}
```

Concrete example — create worklog:

```
pub async fn execute(&self, command: CreateWorklogCommand) ->
UseCaseResult<CreateWorklogResponse> {
    command.validate()?;
    let worklog = command_to_worklog(command)?;
    let id = worklog.id();
    self.repository.save(&worklog).await?;
    Ok(CreateWorklogResponse::new(id))
}
```

8.4 Login use case (multi-repository)

```
pub struct LoginUseCase<R, U> {
    refresh_token_repository: R,
```

```

    user_repository: U,
    jwt_config: JwtConfig,
}

```

Rust lesson: **multiple type parameters** when a use case needs several ports. Bounds appear on the `impl` block:

```

impl<R, U> LoginUseCase<R, U>
where
    R: RefreshTokenRepository,
    U: UserRepository,
{ ... }

```

Flow:

1. Trim login; reject empty credentials (generic `InvalidCredentials` — no user enumeration).
2. Find user by email **or** name.
3. Check `user.is_active()`.
4. `verify_password` (argon2).
5. Issue access JWT + store hashed refresh token.

8.5 Auth helpers

Piece	Crate	Role
Password hashing	argon2 + password-hash	Store/verify password hashes
Device token hashing	sha2	Hash <code>wl_...</code> tokens before DB lookup
JWT	jsonwebtoken	Sign/validate access tokens

JwtConfig:

```

pub struct JwtConfig {
    pub secret: String,
    pub access_ttl_secs: i64,
    pub refresh_ttl_secs: i64,
}

```

8.6 Excel export

`ExportWorklogsUseCase` filters worklogs (large page size) and builds an `.xlsx` via `rust_xlsxwriter` with title row, frozen header, autofilter, styled tags. Both API and TUI reuse this path (API returns bytes; TUI writes a file via SDK download).

8.7 Public re-exports

`use_cases/src/lib.rs` re-exports commands, responses, and use-case types so `api` and `sdk` can write:

```
use use_cases::{CreateWorklogUseCase, FilterWorklogsCommand, ...};
```

When rebuilding: keep one file per use case; keep DTOs separate from entities.

9. The infrastructure crate (PostgreSQL + sqlx)

Path: infrastructure/

Role: Implement domain repository traits with PostgreSQL.

9.1 Key dependency: sqlx

```
sqlx = { version = "0.8", features = [
    "runtime-tokio", "postgres", "chrono", "uuid", "macros", "migrate"
] }
```

sqlx is an async SQL toolkit with optional **compile-time query checking** against a live database (or offline `.sqlx` cache).

9.2 Connection pool

```
pub async fn connect(database_url: &str) -> Result<PgPool, sqlx::Error> {
    PgPoolOptions::new()
        .max_connections(5)
        .connect(database_url)
        .await
}
```

`PgPool` is cheap to clone (internally reference-counted). Repositories hold a pool and run queries on it.

9.3 Implementing a repository

```
pub struct PostgresWorklogRepository {
    pool: PgPool,
}

impl WorklogRepository for PostgresWorklogRepository {
    async fn get(&self, user_id: UserId, id: WorklogId) ->
    RepositoryResult<Worklog> {
        let row = sqlx::query_as::<_, WorklogRow>(<SQL *>)
            .bind(id.as_uuid())
            .bind(user_id.as_uuid())
            .fetch_optional(&self.pool)
            .await
            .map_err(|_| RepositoryError::QueryFailed)?;
        // map row → domain entity via reconstitute
    }
}
```

Pattern:

1. SQL returns a **row DTO** (`WorklogRow`) with `sqlx`-friendly types.
2. A **mapper** converts `row` → domain (`Worklog::reconstitute(...)`).

3. Map sqlx errors into RepositoryError so domain/use cases do not depend on sqlx error types.

9.4 Filter SQL shape

Filters use nullable bind parameters:

```
AND ($4::text[] IS NULL OR tags && $4)
AND ($6::text IS NULL OR description ILIKE '%' || $6 || '%')
```

PostgreSQL && = array overlap (any tag matches). Soft-deleted rows are excluded with deleted_at IS NULL.

Statistics (total duration, days worked) are computed in SQL CTEs alongside pagination — see WORKLOG_FILTER_WITH_STATS in worklog_repository.rs.

9.5 Migrations

Located in infrastructure/migrations/:

File	Purpose
20240528000001_create_worklogs.sql	worklogs table, GIN index on tags
20240612000001_add_users_and_tokens.sql	users, api_tokens, backfill user_id, legacy user
20240613000001_add_refresh_tokens.sql	refresh_tokens for JWT refresh rotation

Apply:

```
sqlx migrate run --source infrastructure/migrations
```

Schema highlights for worklogs:

- id UUID PRIMARY KEY
- datetime TIMESTAMPTZ
- duration INTERVAL with CHECK (duration > 0)
- tags TEXT[]
- deleted_at for soft delete
- user_id FK to users

9.6 Offline mode

Checked-in query metadata under .sqlx/ allows CI builds without Postgres:

```
SQLX_OFFLINE=true cargo build
# after SQL changes:
cargo sqlx prepare --workspace
```

9.7 Recreate checklist

1. Write migrations first.
2. Create row structs + FromRow(sqlx).
3. Implement each trait method.
4. Map RepositoryError::NotFound vs QueryFailed.

5. Keep SQL user-scoped.

10. The api crate (Axum HTTP server)

Path: api/

Role: Inbound HTTP adapter. Translate JSON $\leftrightarrow$ commands; run use cases; map errors to status codes.

10.1 Stack

Crate	Role
axum 0.8	HTTP framework on hyper/tokio
tokio	Async runtime
tower-http	TraceLayer request logging
tracing / tracing-subscriber	Structured logs
serde / serde_json	JSON bodies

10.2 main bootstrap sequence

1. Init tracing subscriber (RUST_LOG / default api=debug).
2. Read DATABASE_URL, connect pool.
3. Load JwtConfig from JWT_SECRET (must be $\geq$ 32 chars).
4. Wrap Postgres repos in Arc.
5. Build AppState (all use cases pre-constructed).
6. Build router; bind HOST/PORT (default 127.0.0.1:3000).
7. `axum::serve(listener, app).await`.

10.3 AppState

Holds every use case behind an inner Arc:

```
pub struct AppState {  
    inner: Arc<Inner>,  
}
```

Handlers call `state.create_worklog().execute(command).await?`.

Rust lesson: Axum requires state to be Clone + Send + Sync. Arc makes cloning cheap.

10.4 Router

```
let protected = Router::new()  
    .route("/worklogs", post(create).get(filter_query))  
    .route("/worklogs/filter", post(filter))  
    .route("/worklogs/export", get(export_query).post(export))  
    .route("/worklogs/{id}", get(get).put(edit).delete(delete))  
    .route_layer(from_fn_with_state(state.clone(), require_auth));
```

```
Router::new()
```

```

.route("/health", get(health))
.route("/auth/login", post(login))
.route("/auth/refresh", post(refresh))
.route("/auth/logout", post(logout))
.merge(protected)
.layer(TraceLayer::new_for_http());

```

Public: health + auth. Protected: everything under /worklogs/.*

10.5 Handler anatomy

```

pub async fn create(
  State(state): State<AppState>,
  Extension(actor): Extension<ActorContext>,
  Json(body): Json<CreateWorklogRequest>,
) -> ApiResult<(StatusCode, Json<CreateWorklogJson>)> {
  let command = create_worklog_request_to_command(body, actor.user_id());
  let response = state.create_worklog().execute(command).await?;
  Ok((StatusCode::CREATED, Json(worklog_id_to_json(response.id))))
}

```

Axum **extractors** (function arguments):

Extractor	Meaning
State<AppState>	Shared app state
Json<T>	Deserialize body
Extension<ActorContext>	Value inserted by middleware
Path / Query	URL parameters

Return type `ApiResult<...>` maps `UseCaseError` → HTTP JSON error responses (400, 401, 404, 500).

10.6 DTO vs command vs domain

Three shapes exist on purpose:

1. **HTTP request DTO** (`CreateWorklogRequest`) — serde field names, optional JSON quirks.
2. **Command** (`CreateWorklogCommand`) — application input.
3. **Entity** (`Worklog`) — domain.

Mappers live under `api/src/mapper/`. Never leak sqlx types into JSON.

10.7 Docker

`api/Dockerfile` builds the API image. Migrations are **not** auto-applied in the container — run them externally, then point `DATABASE_URL` at Postgres.

11. Authentication: device tokens and JWT

Worklogger supports **two** bearer schemes on the same `Authorization: Bearer ...` header.

11.1 Device tokens (wl...)

- Minted by **admin CLI** (create-token).
- Plaintext shown **once**; only SHA-256 hash stored in api_tokens.
- TUI/SDK use these for long-lived machine auth.
- Middleware: if token starts with wl_, run AuthenticateTokenUseCase.

11.2 JWT access + refresh

- User logs in with email/username + password → access_token (JWT) + refresh_token (rt...).
- Access TTL default 900s; refresh TTL default 30 days.
- Refresh **rotates** (old refresh revoked, new issued).
- Logout revokes refresh token.

Middleware branch:

```
let actor = if token.starts_with("wl_") {
    state.authenticate_token().execute(&token).await?
} else {
    state.authenticate_jwt().execute(&token).await?
};
request.extensions_mut().insert(actor);
```

11.3 Security practices visible in code

Practice	Where
Never store raw device tokens	hash in DB
Passwords via argon2	use_cases auth
Generic auth failures	InvalidCredentials
JWT secret length check	api startup panic if < 32
Admin CLI gated	WORKLOGGER_ADMIN_TOKEN env

11.4 Recreate auth safely

1. Implement password hash/verify first.
2. Add users + tokens migrations.
3. Device token mint path (admin).
4. JWT issue/validate.
5. Middleware last — wire actor into commands.

12. The sdk HTTP client

Path: sdk/

Role: Typed client so the TUI (or any Rust consumer) does not hand-roll HTTP + JSON.

12.1 Dependencies

Crate	Role
request	HTTP client (json, rustls-tls)
url	Safe URL joining
serde / serde_json	Bodies
thiserror	SdkError
wiremock (dev)	HTTP mocking in tests

12.2 Builder pattern

```
let client = WorkloggerClient::builder()
    .base_url(base_url)
    .token(token)
    .build()?;
```

Rust lesson: builders are idiomatic for optional/config-heavy construction.
build() validates URL and returns Result.

12.3 Client methods (mirror API)

Method	HTTP
health	GET /health
create_worklog	POST /worklogs
filter_worklogs	POST /worklogs/filter
get_worklog	GET /worklogs/{id}
edit_worklog	PUT /worklogs/{id}
delete_worklog	DELETE /worklogs/{id}
export_worklogs	export endpoints → bytes + row count

All authenticated calls use a helper like `authed(request)` that sets `Authorization: Bearer {token}`.

12.4 Error mapping

```
pub enum SdkError {
    Validation(String),
    Network(...),
    Server(String),
    NotFound,
    Unauthorized,
    InvalidResponse(String),
    // ...
}
```

SDK validates commands **before** sending (fail fast). On response, maps status codes and JSON error bodies.

12.5 Why SDK re-exports use_case DTOs

```
pub use use_cases::{
    CreateWorklogCommand, FilterWorklogsCommand, ...
};
```

The TUI builds the same command types the API understands conceptually — the SDK serializes them to JSON. That keeps one vocabulary across layers.

12.6 Recreate tips

1. Start with health + create + filter.
2. Centralize handle_response.
3. Add wiremock tests for 201/400/401/404.

13. The tui terminal UI (Ratatui)

Path: tui/

Role: Interactive terminal front end. **No direct DB access** — only sdk.

13.1 Stack

Crate	Role
ratatui	Immediate-mode TUI widgets (tables, paragraphs, layouts)
crossterm	Raw mode, key events, alternate screen
sdk	API client
shlex	Parse search bar tokens with quotes
jalali-rs / chrono-tz	Display dates in Tehran/Jalali
tokio	Runtime for async HTTP inside sync event loop

13.2 Architecture: Elm / MVU

The TUI is explicitly documented as **Elm-style**:

```
Input (key) → Msg → update(model, msg) → Model'
                        ↓
                    view(model) → widgets
```

Key types:

```
pub enum Mode {
    Normal, Search, AddModal, EditModal, DeleteModal, OpenModal,
}

pub enum Msg {
    Tick,
    Quit,
    Table(table:Msg),
    Search(search_bar:Msg),
}
```

```

    Add(add::Msg),
    Delete(delete::Msg),
    Edit(edit::Msg),
    Open(open::Msg),
}

```

```
pub enum Outcome { Continue, Quit }
```

Each dialog owns its own Model, Msg, update, view under `tui/src/dialogs/` and `components/`.

13.3 Startup and shutdown

```

fn main() -> io::Result<()> {
    let rt = tokio::runtime::Runtime::new()?;
    let mut terminal = setup_terminal()?; // raw mode + alternate screen
    let app = rt.block_on(wiringup()?); // env + health check + load
page
    let result = run_terminal(&mut terminal, &handle, app);
    restore_terminal(&mut terminal)?; // always restore terminal!
    result
}

```

Critical UX rule: always restore the terminal (leave alternate screen, disable raw mode) even on error — otherwise the user’s shell is left broken.

wiringup requires:

- WORKLOGGER_BASE_URL
- WORKLOGGER_TOKEN

13.4 Main loop (conceptual)

1. Draw `ui::view`.
2. Poll crossterm events with a short timeout (allows cursor blink Tick).
3. Map key → Msg via `from_key` (depends on Mode).
4. update may `block_on` async SDK calls (reload table, create, delete, export).
5. Repeat until `Outcome::Quit`.

13.5 Layout

`ui.rs` splits the screen vertically:

1. Title bar (version, mode, counts, duration stats, page)
2. Table (main content)
3. Search bar
4. Help bar

Modals clear the middle and draw dialog widgets on top. Status toasts appear above the search bar.

13.6 Search DSL

Parsed in `search_dsl.rs` using `shlex` for quoted strings. Tokens:

Token	Example
tag: / -tag:	tag:rust,dev
desc:	desc:"code review"
date:	date:1403/01/01..1403/01/31, >=, <=
duration:	duration:1h..4h
id:	UUID

Result is a `FilterWorklogsCommand` sent through the SDK.

13.7 Keybindings (product behavior)

Key	Action
j/k or arrows	Navigate
/	Search
n/a	Add
o	Open detail
d	Delete
e	Export xlsx
q / Ctrl+C	Quit

13.8 Display model vs domain

```
pub struct WorklogRow {
    pub id: Uuid,
    pub date: String,          // already formatted Jalali
    pub duration: String,      // "2h30m"
    pub description: String,
    pub tags: String,
}
```

The TUI converts API/domain objects into **view models** for rendering. Keep formatting in `format.rs`.

13.9 Recreate a minimal Ratatui app

1. Enable raw mode + alternate screen.
2. Loop: draw a Paragraph, read a key, quit on q.
3. Add Table + TableState.
4. Introduce Mode enum + modal overlays.
5. Only then wire SDK.

14. The admin CLI (Clap)

Path: admin/

Binary name: worklogger-admin

14.1 Why a separate binary

User/token provisioning needs DB access and a shared admin secret. End users run the TUI against the API with a device token; admins run this CLI on a trusted machine.

14.2 Clap derive API

```
#[derive(Parser)]
#[command(name = "worklogger-admin")]
struct Cli {
    #[command(subcommand)]
    command: Command,
}

#[derive(Subcommand)]
enum Command {
    CreateUser { #[arg(long)] name: String, #[arg(long)] email:
Option<String>, ... },
    CreateToken { #[arg(long)] user: Uuid, #[arg(long)] label: String },
    // ...
}
```

Rust lesson: Clap's derive macros generate argv parsing from structs/enums — similar to Python argparse or Node commander, but typed.

14.3 Commands

Subcommand	Effect
create-user	Insert user; optional password
set-password	Reset argon2 hash
create-token	Print wl... once
create-legacy-token	Token for bootstrap user
revoke-token	Revoke by token id
disable-user / enable-user / delete-user	Lifecycle

Requires DATABASE_URL + WORKLOGGER_ADMIN_TOKEN.

14.4 Exit codes

```
async fn main() -> ExitCode {
    if let Err(message) = run().await {
        eprintln!("error: {message}");
        return ExitCode::FAILURE;
    }
    ExitCode::SUCCESS
}
```

Prefer ExitCode over Result at the binary edge for clear process status.

15. End-to-end data flows

15.1 Create worklog (TUI → DB)

User presses ``n``, fills fields, Enter
↓
tui dialog builds `CreateWorklogCommand`
↓
`sdk.WorkloggerClient::create_worklog`
↓ HTTP POST /worklogs + Bearer wl_...
api middleware → `AuthenticateToken` → `ActorContext`
↓
controller maps JSON → `CreateWorklogCommand` (`user_id` from actor)
↓
`CreateWorklogUseCase::execute`
↓ `validate` + `command_to_worklog` (domain VOs)
`PostgresWorklogRepository::save`
↓
`INSERT INTO worklogs (...)`
↓
`201 { "id": "..." }` → TUI reloads table via filter

15.2 Filter / search

Search bar DSL → `FilterWorklogsCommand`
↓
`POST /worklogs/filter`
↓
`FilterWorklogsUsecase` → `criteria` → `repository.filter`
↓
`SQL WHERE` + pagination + stats CTE
↓
`PageResult` + statistics → table rows

15.3 JWT login (curl / future web client)

`POST /auth/login { login, password }`
↓
`LoginUseCase` → `verify argon2` → `issue JWT` + refresh
↓
Authorization: Bearer <access_token> on /worklogs
↓
middleware `authenticate_jwt` → `ActorContext`

15.4 Soft delete

Domain sets `deleted_at`; repository update/delete marks row. Filters always skip `deleted_at IS NOT NULL`. Data remains for audit.

15.5 Export

Same filters as list, higher size cap (up to 100k). Use case builds `xlsx` bytes. API returns file download headers; TUI writes under `WORKLOGGER_EXPORT_DIR` (default `~/Download`).

16. Framework and crate catalog

Use this as a shopping list when rebuilding or explaining the stack to teammates.

16.1 Language & tooling

Tool	Purpose
rustc	Compiler
cargo	Build, test, deps
rustup	Toolchain manager
Edition 2024	Language edition in all packages
sqlx-cli	Migrations + prepare

16.2 Web / async

Crate	Used in	Purpose
tokio	api, tui, infra, admin, sdk tests	Async runtime
axum	api	HTTP routing, extractors, middleware
tower-http	api	HTTP middleware (tracing)
hyper	(via axum)	HTTP implementation
request	sdk	Outbound HTTP
tracing	api	Diagnostics

16.3 Data

Crate	Used in	Purpose
sqlx	infrastructure	Postgres access
chrono	many	Date/time
chrono-tz	use_cases, tui, common	Time zones (Tehran)
uuid	many	IDs
serde / serde_json	api, sdk, common	Serialization

16.4 Domain / security / export

Crate	Purpose
thiserror	Error enums
argon2 / password-hash	Password hashing
sha2	Device/refresh token hashing
jsonwebtoken	JWT
rust_xlsxwriter	Excel export
jalali-rs	Jalali calendar

16.5 UI / CLI

Crate	Purpose
ratatui	TUI widgets
crossterm	Terminal control
shlex	Shell-like tokenization for search
clap	Admin CLI

16.6 Testing helpers

Crate	Purpose
wiremock	Mock HTTP for sdk tests

16.7 Mapping to other ecosystems

Worklogger	Node	Python	Java
Axum	Express/Fastify	FastAPI	Spring WebFlux
sqlx	Prisma/Knex	SQLAlchemy async	jOOQ / Spring Data JDBC
Tokio	libuv / async	asyncio	Project Reactor
Ratatui	blessed/ink	textual/urwid	—
Clap	yargs/commander	click/argparse	picocli
Serde	zod + JSON	pydantic	Jackson

17. Rebuild blueprint: duplicate the project

This section is the **recipe**. Follow it in order; each step should compile before moving on.

Phase 0 — Empty workspace

```
mkdir worklogger && cd worklogger
cat > Cargo.toml << 'EOF'
[workspace]
members = ["common", "core", "use_cases", "infrastructure", "api", "sdk",
"tui", "admin"]
resolver = "2"
EOF
```

Create each package:

```
cargo new --lib common
cargo new --lib core
cargo new --lib use_cases
cargo new --lib infrastructure
cargo new --lib sdk
cargo new api
cargo new tui
cargo new admin
```

Wire path dependencies in each Cargo.toml (see sections above). Set edition = "2024" (or 2021 if your toolchain is older — adjust).

Phase 1 — common

1. Implement PagingParams + PageResult<T>.
2. Implement filter structs with Serde derives.
3. cargo test -p common.

Phase 2 — core

1. DomainError + DomainResult.
2. Value objects: UserId, WorklogId, Email, Tag, Tags, Description, WorklogDuration, timestamps.
3. Entities: Worklog, User, ApiToken with unit tests for state transitions.
4. Traits: WorklogRepository, UserRepository, TokenRepository, RefreshTokenRepository.
5. ActorContext, filter criteria types.

Checkpoint: cargo test -p core with zero sqlx/axum deps.

Phase 3 — use_cases (in-memory fake)

1. Define commands/responses.
2. Implement CreateWorklogUseCase against a **fake** in-memory repository in tests (a struct implementing the trait with a Mutex<HashMap<...>>).
3. Add validation + mappers + Jalali parsing.
4. Add remaining worklog use cases.
5. Add auth use cases + argon2/JWT.
6. Add export last.

Checkpoint: use cases tested without Postgres.

Phase 4 — infrastructure

1. Write SQL migrations matching entities.
2. connect pool helper.
3. Implement Postgres repositories.
4. Integration test against local Postgres (optional but recommended).
5. sqlx migrate run + cargo sqlx prepare.

Phase 5 — api

1. AppState wiring.
2. /health.
3. Auth middleware + login routes.
4. Worklog CRUD/filter/export controllers.
5. Error → HTTP mapping.
6. Manual curl tests from README.

Phase 6 — sdk

1. Builder + health.
2. Create/filter/delete.
3. Wiremock tests.
4. Export bytes helper.

Phase 7 — tui

1. Terminal setup/teardown.
2. Static fake data table.
3. Elm Msg/update loop.
4. Wire SDK + env vars.
5. Search DSL + dialogs + export.

Phase 8 — admin

1. Clap subcommands.
2. Gate on WORKLOGGER_ADMIN_TOKEN.
3. Call create-user / create-token use cases.

Phase 9 — Polish

1. .env.example, README, Docker for API.
2. Tracing, graceful error messages.
3. Soft-delete consistency across layers.
4. Pagination stats in title bar.

Minimal viable clone (if time-boxed)

If you cannot rebuild everything, rebuild this slice first:

1. core Worklog + duration/tags VOs + repository trait
2. In-memory use case create/filter
3. sqlx save/filter

4. Axum POST/GET
5. One-screen Ratatui list

That already teaches 80% of the architecture.

Suggested file ownership when working as a team

Person	Owns
Domain-focused	core, common
App logic	use_cases
DBA/backend	infrastructure, migrations
API	api
Client	sdk, tui
Ops	admin, Docker, env

18. Exercises and next steps

Work through these to lock in learning. Each maps to a real extension of Worklogger.

Exercise A — New value object

Add a `ProjectCode` value object (non-empty, max 16 chars, alphanumeric). Attach optional project to `Worklog`. Propagate through migration, repository, API DTO, SDK, TUI add dialog.

Teaches: newtype validation + cross-layer change discipline.

Exercise B — In-memory repository

Implement `InMemoryWorklogRepository` in `use_cases` tests. Run `create/filter/delete` without Postgres.

Teaches: traits as seams; why hexagonal architecture pays off.

Exercise C — New API endpoint

GET `/worklogs/stats/tags` returning tag histogram for the actor. Reuse filter criteria.

Teaches: Axum routes + SQL aggregation + auth scoping.

Exercise D — TUI mode

Add a “stats” mode showing totals by tag. New `Mode`, `Msg` variant, view.

Teaches: Elm architecture extension without rewriting the loop.

Exercise E — Replace JWT library (thought experiment)

List every file that would change if you swapped `jsonwebtoken` for another crate. Confirm core stays untouched.

Teaches: dependency direction.

Exercise F — Explain ownership aloud

Pick `CreateWorklogUseCase::execute` and narrate what is owned vs borrowed at each line. If you can do this, you understand Rust's core model.

19. Glossary

Term	Meaning in this project
Crate	A Rust library or binary package
Workspace	Multiple crates sharing one lockfile
Trait	Interface / contract
Port	Trait defined by domain for outside world
Adapter	Implementation of a port (Postgres, HTTP, CLI)
Entity	Domain object with identity (Worklog, User)
Value object	Validated immutable-ish wrapper (Email, Tags)
Aggregate	Cluster of entities treated as a unit (here: Worklog)
DTO / Command	Data crossing a layer boundary
Use case	Application service for one user intention
Soft delete	Mark <code>deleted_at</code> instead of removing row
Actor	Authenticated caller (ActorContext)
Elm architecture	Model-View-Update UI loop
Newtype	<code>struct Email(String)</code> wrapper
Arc	Shared ownership smart pointer
Extractor	Axum handler argument that pulls request data
Migration	Versioned SQL schema change
Jalali	Persian calendar used for display/input

Appendix A — Environment variables cheat sheet

Variable	Consumed by	Required?	Notes
DATABASE_URL	api, admin, sqlx	yes for those	Postgres URL
JWT_SECRET	api	yes	≥ 32 characters
JWT_ACCESS_TTL_SECS	api	no	default 900
JWT_REFRESH_TTL_SECS	api	no	default 2592000
HOST / PORT	api	no	bind address
WORKLOGGER_BASE_URL	tui	yes	e.g. <code>http://127.0.0.1:3000</code>
WORKLOGGER_TOKEN	tui	yes	<code>wl_...</code> device token
WORKLOGGER_EXPORT_DIR	tui	no	default <code>~/Download</code>
WORKLOGGER_ADMIN_TOKEN	admin	yes	shared secret gate
SQLX_OFFLINE	build	no	true for offline compile
RUST_LOG	api	no	tracing filter

Appendix B — API surface summary

Method	Path	Auth	Purpose
GET	<code>/health</code>	no	Liveness
POST	<code>/auth/login</code>	no	JWT + refresh
POST	<code>/auth/refresh</code>	no	Rotate refresh
POST	<code>/auth/logout</code>	no	Revoke refresh
POST	<code>/worklogs</code>	yes	Create
GET	<code>/worklogs</code>	yes	Filter (query)
POST	<code>/worklogs/filter</code>	yes	Filter (JSON)
GET/POST	<code>/worklogs/export</code>	yes	XLSX export
GET	<code>/worklogs/{id}</code>	yes	Get one
PUT	<code>/worklogs/{id}</code>	yes	Edit
DELETE	<code>/worklogs/{id}</code>	yes	Soft delete

Appendix C — Rust syntax field guide (cheat sheet)

```
// Binding
let x = 1;
let mut y = 2;
```

```

// Function
fn add(a: i32, b: i32) -> i32 { a + b }

// Async
async fn fetch() -> Result<String, Error> { Ok("ok".into()) }

// Struct + impl
struct Dog { name: String }
impl Dog {
    fn bark(&self) { println!("{}", self.name); }
}

// Enum + match
enum Ip { V4(u8,u8,u8,u8), V6(String) }
match ip { Ip::V4(...) => {}, Ip::V6(s) => {} }

// Result operators
let v = might_fail()?;
let v = might_fail().map_err(MyError::from?);

// Trait
trait Save { async fn save(&self) -> Result<(), Error>; }

// Module
mod inner { pub fn f() {} }
use inner::f;

// Ownership
fn take(s: String) {}           // moves
fn borrow(s: &String) {}        // shared borrow
fn borrow_mut(s: &mut String) {} // exclusive borrow

```

Appendix D — How to study the real repo in order

Read source files in this sequence (roughly increasing “outer”):

1. core/src/error.rs
2. core/src/value_objects/worklog_duration.rs + email.rs + tags.rs
3. core/src/entities/worklog.rs + user.rs
4. core/src/traits/worklog_repository.rs
5. use_cases/src/dtos/commands/create_worklog.rs
6. use_cases/src/use_cases/create_worklog.rs
7. use_cases/src/use_cases/login.rs
8. infrastructure/migrations/*.sql
9. infrastructure/src/postgres/worklog_repository.rs (skim SQL)
10. api/src/main.rs + state.rs + routes/routers.rs
11. api/src/middleware/auth.rs + routes/controllers/worklog_create.rs
12. sdk/src/client.rs (first 120 lines)
13. tui/src/main.rs + message.rs + app.rs + ui.rs
14. admin/src/main.rs

If you can explain each file's responsibility in one sentence, you can recreate the system.

Closing

Worklogger is not a toy CRUD demo: it is a **small production-shaped Rust workspace**. The durable lessons are:

1. **Keep domain pure** — traits in, frameworks out.
2. **Validate at boundaries** — commands and value objects.
3. **One use case = one intention** — easy to test and reuse from API, CLI, or future UIs.
4. **Speak Rust through types** — Result, newtypes, and enums replace nulls and stringly errors.
5. **UI is just another adapter** — Ratatui and Axum both call the same application layer (directly or via HTTP/SDK).

With this tutorial and the repository side by side, a developer from another ecosystem should be able to **duplicate the architecture**, implement the same crates, and grow fluent in Rust by *using* it where it matters — in a real design — rather than only in isolated language exercises.

Document version: 1.0 — aligned with Worklogger workspace layout (core, common, use_cases, infrastructure, api, sdk, tui, admin).

20. Deep dive: implementing create-worklog across every layer

This chapter walks through **one feature** in full detail — creating a worklog — as if you were implementing it yourself. If you only study one vertical slice, study this.

20.1 Domain: invent the type before the database

Start with the question: *What is a worklog, independent of Postgres or HTTP?*

Required facts:

- Identity (WorklogId UUID)
- Owner (UserId)
- When the work happened (WorklogDateTime)
- How long (WorklogDuration)
- Classification (Tags)
- Human summary (Description)
- Audit fields (created_at, updated_at, deleted_at)

You encode rules in constructors, not in comments:

```
duration ∈ (0, 24h)
tags length ∈ [1, 50]
description non-empty after trim
```

Pseudo-implementation order:

1. Write failing unit tests for `WorklogDuration::try_from_secs(0)` and `try_from_secs(86_400)`.
2. Implement duration VO until tests pass.
3. Same for Tag / Tags / Description.
4. Implement `Worklog::create` that composes VOs and sets timestamps with `chrono::Utc::now()`.
5. Implement `soft_delete` tests (second delete must error).

Rust lesson — privacy as design: fields are private so the only way to get an invalid `Worklog` is to bypass the module (you cannot from outside the crate). That is stronger than a Python dataclass with a `__post_init__` people might skip.

20.2 Application: the command is not the entity

HTTP and TUI should not construct `Worklog` directly. They send a **command**:

```
CreateWorklogCommand {
    user_id,           // from auth, never from client body in API
    jalali_date?,      // optional string in user calendar
    duration_secs,     // raw u64 from forms/JSON
    tags: Vec<String>, // raw strings
    description,       // raw string
}
```

Why strings at the edge?

- UI and JSON are stringly by nature.
- Parsing Jalali dates needs `jalali-rs` + `timezone` — that is application concern, not pure domain identity.
- Validation errors can be listed for forms (vec of messages on filter commands).

Mapper responsibilities (`command_to_worklog`):

1. Parse Jalali → UTC `DateTime` (or “today in Asia/Tehran”).
2. `WorklogDuration::try_from_secs`.
3. `Tags::try_from_strs`.
4. `Description::try_new`.
5. `Worklog::create(user_id, datetime, duration, tags, description)`.

If any step fails, return `UseCaseError` — do not save partial state.

20.3 Persistence: save means INSERT of reconstituted facts

The repository receives `&Worklog` and writes columns. Typical column mapping:

Domain	SQL
id	UUID
user_id	UUID
datetime	TIMESTAMPZ
duration	INTERVAL (from seconds)
tags	TEXT[]
description	TEXT
audit stamps	TIMESTAMPZ

On read, you **must** use `reconstitute`, not `create`, or you would mint new IDs/timestamps and destroy history.

20.4 API: extractors do the boring work

Handler responsibilities (and *only* these):

1. Authenticated `ActorContext` already in extensions.
2. Deserialize JSON body.
3. Map to command with `actor.user_id()`.
4. `execute`.
5. Map response to JSON + 201.

Anti-patterns to avoid when cloning:

- Accepting `user_id` from JSON (privilege escalation).
- Running SQL in the controller.
- Returning `sqlx` errors verbatim to clients.

20.5 SDK: same command, different transport

The TUI builds `CreateWorklogCommand` locally (with a placeholder/`nil` user id if the API ignores it and uses the token's user). SDK:

1. `command.validate()` locally.
2. Serialize a JSON body the API expects.
3. `POST` with bearer token.
4. Parse `{ "id": "..."}` into `CreateWorklogResponse`.

20.6 TUI: dialog state machine

Add dialog typically has:

- field list (date, duration, description, tags)
- focus index
- draft strings

- validation error string

Keys:

- Tab / Shift+Tab — move focus
- printable chars — edit focused field
- Enter — submit (async create + reload)
- Esc — cancel, back to Mode::Normal

After success, set a short-lived `status_message` toast.

20.7 Checklist you can print

- ☐ Domain VO tests green
 - ☐ `Worklog::create / soft_delete` tests green
 - ☐ Command validate covers empty tags/description/duration
 - ☐ Use case saves via trait (fake repo in unit test)
 - ☐ Migration includes constraints
 - ☐ Postgres save + get round-trip
 - ☐ API returns 201 and 400 appropriately
 - ☐ SDK mirrors create
 - ☐ TUI can add without leaving terminal broken on panic (restore in finally equivalent)
-

21. Deep dive: filtering, search DSL, and SQL

Filtering is the second vertical slice worth mastering. It touches DSL parsing, DTO mapping, criteria objects, and dynamic SQL binds.

21.1 Layers of filter representation

```
TUI string → FilterWorklogsCommand → WorklogFilterCriteria → SQL binds
API JSON   ↗
API query  ↗
```

Do not let SQL strings leak into the TUI. Do not let Ratatui know about `ILIKE`.

21.2 Command validation vs criteria

The command may contain human strings (`duration_from: "1h"`). Validation parses them into typed bounds. Criteria should already be typed for the repository (epochs, dates, string arrays).

21.3 Why nullable binds beat string-built SQL

```
AND ($6::text IS NULL OR description ILIKE '%' || $6 || '%')
```

When the user did not filter by description, bind `NULL` and the clause becomes a no-op. Benefits:

- One SQL statement shape (easier for `sqlx` offline cache).
- Fewer injection risks than concatenating snippets.
- Plan caching friendlier than dozens of variants.

21.4 Array overlap for tags

PostgreSQL tags `&& $4` means “arrays overlap.” That matches product language: `tag:rust,dev` → entries having **any** of those tags. Exclusion uses `NOT (tags && excluded)`.

21.5 Search DSL parsing strategy

1. `shlex::split` to respect quotes.
2. For each token, split on first `:` into (key, value).
3. Match key (tag, -tag, desc, date, duration, id).
4. For ranges, split on `..` or detect `>=` / `<=` prefixes.
5. Accumulate into command fields; on unknown token, surface a UI error.

Rust lesson: prefer returning `Result` from the parser; the UI displays `Err` as a toast instead of panicking.

21.6 Pagination math

```
offset = (page.saturating_sub(1)) * size
total_pages = ceil(total_items / page_size)
```

TUI keeps `current_page` in `App` and rewrites `command.paging` before each fetch. After delete, reload the same page (or clamp if empty).

21.7 Statistics

Title bar shows total duration and days worked for the **current filter**, not only the current page. Those aggregates belong in SQL (`SUM`, `COUNT(DISTINCT date)`) over the filtered set, returned beside the page of rows (`WorklogFilterResult` / response statistics).

22. Deep dive: ownership worked examples from this repo

Non-Rust developers often understand architecture but stumble on the borrow checker. These examples are taken from `Worklogger` patterns.

22.1 Why `save` takes `&Worklog`

```
async fn save(&self, worklog: &Worklog) -> RepositoryResult<()>
```

If it took `Worklog` by value, the use case would **move** the entity into `save` and could not read `worklog.id()` afterward without cloning first. Borrowing keeps ownership in the use case.

22.2 Why getters return references for large fields

```
pub fn tags(&self) -> &Tags { &self.tags }
pub fn name(&self) -> &UserName { &self.name }
```

Copying a `Vec` of tags on every get would be wasteful. IDs are copy (UUID wrappers), so they return by value:

```
pub fn id(&self) -> WorklogId { self.id }
```

Rule of thumb in this codebase: small Copy value objects return by value; owned collections return &.

22.3 Arc and cloning state

```
let worklog_repo = Arc::new(PostgresWorklogRepository::new(pool.clone()));
```

- `pool.clone()` clones the pool handle (not every connection).
- `Arc::new` wraps the repo.
- `AppState` clones share the same inner use cases.

You need this because Axum runs many concurrent tasks; each needs access to repos safely.

22.4 Mutating through `&mut self`

```
pub fn soft_delete(&mut self) -> DomainResult<()>
```

Callers need a mutable binding:

```
let mut worklog = repo.get(...).await?;  
worklog.soft_delete()?;  
repo.update(&worklog).await?;
```

Immutable `let worklog` would not compile — the compiler forces you to acknowledge mutation.

22.5 Lifetimes you mostly do not write

Many tutorials scare people with 'a lifetimes. In Worklogger, most APIs avoid returning references tied to locals; they return owned DTOs (`CreateWorklogResponse`, `String` errors). That is intentional ergonomics.

When you *do* see references, they are short-lived (`&str` parameters, `&Worklog` borrows for a single `await` call).

22.6 `async` and holding locks across `await`

If you implement an in-memory fake repo with `Mutex<HashMap<...>>`:

```
// Prefer: lock, clone data, drop guard, then await other things  
let snapshot = {  
    let guard = self.inner.lock().unwrap();  
    guard.get(&id).cloned()  
};
```

Do not hold the `MutexGuard` across `.await` in `async` code — it is not Send in the usual `std` mutex pattern and can deadlock under concurrency.

23. Deep dive: error mapping across layers

A single failure can climb several type ladders. Understanding this prevents “stringly” APIs.

23.1 Example: empty description on create

```
Description::try_new("")
  → DomainError::EmptyDescription
  → UseCaseError::Domain(...) // via From
  → ApiError::BadRequest(...) // in api/error.rs mapping
  → HTTP 400 { "error": "...", "details": [...] }
  → SdkError::Validation / Server
  → TUI status toast
```

23.2 Example: wrong password

```
LoginUseCase
  → AuthError::InvalidCredentials
  → UseCaseError::Auth
  → HTTP 401
```

Do not return “user not found” vs “bad password” separately — the code intentionally collapses them.

23.3 Example: missing worklog

```
RepositoryError::NotFound
  → UseCaseError::Repository
  → HTTP 404
```

23.4 Implementing From yourself

thiserror’s #[from] generates:

```
impl From<DomainError> for UseCaseError {
  fn from(e: DomainError) -> Self { Self::Domain(e) }
}
```

That is why ? works seamlessly. When adding a new error variant, add #[from] only when automatic conversion is always correct.

23.5 When to use map_err

```
.map_err(|_| RepositoryError::QueryFailed)?
```

Used when you want to **erase** sqlx details at the infrastructure boundary (do not leak DB internals to API clients). Log the real error with tracing before mapping if you need ops visibility.

24. Testing strategy for a workspace like this

24.1 Pyramid

Level	Where	What
Unit	core entity/VO tests	Invariants, no I/O
Unit	use_cases + fake repos	Orchestration
Contract	sdk + wiremock	HTTP JSON shapes
Integration	infrastructure + real Postgres	SQL correctness
Manual / e2e	curl + TUI	UX and auth

24.2 `#[cfg(test)]` modules

Rust keeps tests next to code:

```
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn user_disable_and_enable() { ... }
}
```

Run with `cargo test -p core`.

24.3 Testing async use cases

```
#[tokio::test]
async fn creates_worklog() {
    let repo = FakeRepo::default();
    let uc = CreateWorklogUseCase::new(repo);
    let resp = uc.execute(sample_command()).await.unwrap();
    assert!(!resp.id.to_string().is_empty());
}
```

24.4 What not to test excessively

- Ratatui pixel layouts (brittle). Prefer testing DSL parser and command validation.
- Generated Clap help text.
- Every sqlx type conversion if mappers are thin and covered by one round-trip test.

25. Operational notes: running, migrating, shipping

25.1 Local development loop

```
# terminal 1
export DATABASE_URL=...
export JWT_SECRET=...
cargo watch -x 'run -p api' # if cargo-watch installed
```

```
# terminal 2
export WORKLOGGER_BASE_URL=...
export WORKLOGGER_TOKEN=...
cargo run -p tui
```

25.2 Migration discipline

1. Add a new timestamped SQL file — never rewrite applied migrations that already ran in shared environments.
2. Run migrate.
3. Update repositories/mappers.
4. `cargo sqlx prepare --workspace`.
5. Commit `.sqlx` cache updates.

25.3 Release binaries

```
cargo build --release -p api -p tui -p admin
# artifacts under target/release/
```

Strip and package as needed for your OS. The TUI is a local client; the API is the service you deploy.

25.4 Docker networking reminder

Inside a container, `localhost` is the container. Use Docker DNS names or `host.docker.internal` for host Postgres. The README covers a shared bridge network example.

25.5 Secrets

Never commit real `.env`. Rotate `JWT_SECRET` and admin token if leaked. Device tokens are bearer secrets — treat `wl_...` like passwords.

26. Design decisions explained (why the code looks this way)

26.1 Why hexagonal instead of “sqlx in the TUI”?

Because Worklogger already has two UIs (TUI + HTTP) and an admin tool. Duplicating SQL would fork business rules. The TUI talking HTTP also mirrors how a future web UI would work.

26.2 Why soft deletes?

Work logs are audit-sensitive. Soft delete lets you restore and keeps historical integrity. Filters hide deleted rows by default.

26.3 Why Jalali in the UI but UTC in the DB?

Storage in UTC is universal. Display/parse in Jalali matches the product audience. Conversion stays in `use_cases` / UI formatting helpers, not in raw SQL date math when avoidable.

26.4 Why both query-param and JSON filter endpoints?

Query params are handy for curl and simple clients; JSON bodies express nested filters more clearly. Both map to the same command/use case.

26.5 Why device tokens and JWT?

- Device tokens: long-lived automation / TUI installs without storing passwords on disk.
- JWT: interactive login for humans, short-lived access, refresh rotation.

26.6 Why `command_user_id()` nil in TUI?

The API overwrites identity from the bearer token. The TUI still constructs commands that include a `user_id` field for type reuse; the server trusts the actor, not the client.

27. Annotated “day in the life” of a contributor

Imagine you join the project and must add **edit worklog** (already present — treat as a study map):

1. Read `EditWorklogCommand` fields and validate.
2. Read `EditWorklogUseCase`: load entity, apply setters, `repository.update`.
3. Confirm entity setters call `touch()`.
4. Find `PostgresWorklogRepository::update` SQL.
5. Find API `PUT /worklogs/{id}` controller + request DTO.
6. Find SDK `edit_worklog`.
7. Find TUI `EditModal` dialog messages.

That traversal order — **command** → **use case** → **entity** → **SQL** → **HTTP** → **SDK** → **UI** — is the standard way to navigate this repo for any feature.

28. Common mistakes when non-Rust developers clone this design

Mistake	Symptom	Fix
Putting sqlx in core	Domain cannot be tested without DB	Depend only inward
Using <code>.unwrap()</code> in libraries	Crashes in production	Return Result
Accepting user id from JSON	Users edit others' data	Take id from ActorContext
Forgetting terminal re-store	Broken shell after TUI panic	<code>restore_terminal</code> in all paths
Building SQL with string format from user input	Injection risk	Bound parameters
Cloning huge structs unnecessarily	Slow TUI	Pass references; clone rows for display model only
Blocking the Tokio runtime with heavy CPU in <code>async</code>	Latency spikes	<code>spawn_blocking</code> for CPU-heavy export if needed
Changing migration files after deploy	Drift between envs	Only add new migrations

29. Extended recreate script (checklist form)

Use this as a printable build sheet.

Week 1 — Language + domain

- ☐ Install rustup, configure toolchain
- ☐ Complete Rust Book ch. 1-6 (ownership, structs, enums) while reading §4 of this tutorial
- ☐ Implement core VOs + `Worklog` with tests
- ☐ Define repository traits

Week 2 — Application + fake adapters

- ☐ Commands/responses
- ☐ Create/Get/Filter/Delete use cases
- ☐ In-memory repository
- ☐ Jalali parse helper

Week 3 — Postgres

- ☐ Migrations
- ☐ sqlx repositories
- ☐ Manual SQL checks in psql

Week 4 — API + auth

- ☐ Axum health + CRUD
- ☐ Device token auth
- ☐ JWT login/refresh
- ☐ Error JSON format

Week 5 — Clients

- ☐ SDK + wiremock
- ☐ Admin CLI
- ☐ Ratatui list + add + search

Week 6 — Hardening

- ☐ Export xlsx
- ☐ Soft delete consistency
- ☐ Docker
- ☐ README / env sample
- ☐ Offline sqlx cache in CI

If you finish this schedule, you have effectively duplicated Worklogger.

30. Final reference: crate-by-crate public surface

core (as domain)

- Entities: Worklog, User, ApiToken, refresh token entity
- VOs: ids, email, name, tags, description, duration, datetime, timestamps
- Traits: worklog/user/token/refresh repositories
- ActorContext, DomainError, filter criteria/results

common

- PagingParams, PageResult<T>
- Filter DTOs

use_cases

- All *UseCase types listed in lib.rs
- Commands/responses
- JwtConfig, password/JWT helpers
- Export use case

infrastructure

- connect

- PostgresWorklogRepository, PostgresUserRepository, PostgresTokenRepository, PostgresRefreshTokenRepository
- migrations

api

- Binary server
- Routes in §Appendix B

sdk

- WorkloggerClient, WorkloggerClientBuilder, SdkError

tui

- Interactive binary; Elm loop; DSL

admin

- worklogger-admin binary; user/token subcommands

End of extended sections (§20–§30). Combined with §1–§19, this tutorial is intended to exceed a 40-page printed teaching length when exported to PDF with standard technical-manual formatting (code blocks, diagrams, and checklists).

31. Side-by-side: the same idea in TypeScript vs Rust

Many readers arrive from TypeScript/Node. This chapter maps familiar patterns to Worklogger’s Rust code so the syntax tax feels smaller.

31.1 Interface vs trait

TypeScript

```
interface WorklogRepository {
  save(worklog: Worklog): Promise<void>;
  get(userId: string, id: string): Promise<Worklog>;
}
```

Rust (Worklogger)

```
pub trait WorklogRepository {
  async fn save(&self, worklog: &Worklog) -> RepositoryResult<()>;
  async fn get(&self, user_id: UserId, id: WorklogId) ->
RepositoryResult<Worklog>;
}
```

Differences that matter:

- No implicit null — absences are Option / Result.
- &Worklog is an explicit borrow.
- IDs are typed (UserId), not bare strings.

31.2 Class vs struct + impl

TypeScript

```
class CreateWorklogUseCase {
  constructor(private repo: WorklogRepository) {}
  async execute(cmd: CreateWorklogCommand): Promise<CreateWorklogResponse>
{ ... }
}
```

Rust

```
pub struct CreateWorklogUseCase<R> { repository: R }
impl<R: WorklogRepository> CreateWorklogUseCase<R> {
  pub fn new(repository: R) -> Self { ... }
  pub async fn execute(&self, command: CreateWorklogCommand) ->
UseCaseResult<...> { ... }
}
```

Generics (<R>) replace constructor DI frameworks. At compile time the compiler monomorphizes for `PostgresWorklogRepository` or `Arc<...>`.

31.3 Zod / class-validator vs newtypes + command.validate

TypeScript often validates with Zod at the HTTP edge. Worklogger validates in two steps: command validation (friendly errors) and domain VO construction (invariants). Both are plain Rust functions — no reflection.

31.4 Express middleware vs Axum middleware

TypeScript

```
app.use(async (req, res, next) => {
  req.actor = await auth(req.headers.authorization);
  next();
});
```

Rust

```
pub async fn require_auth(...) -> Result<Response, ApiError> {
  let actor = /* authenticate */;
  request.extensions_mut().insert(actor);
  Ok(next.run(request).await)
}
```

Handlers then take `Extension<ActorContext>` instead of reading a loosely typed `req.actor`.

31.5 Prisma vs sqlx

Prisma generates a client from a schema. sqlx keeps SQL visible in Rust strings/files and can check it against a live DB at compile time. Worklogger chooses explicit SQL for filter-heavy queries — easier to tune GIN indexes and CTEs.

31.6 React state vs Elm Msg

The TUI is closer to Elm or Redux than to ad-hoc React `useState` sprawl:

keypress → Msg → update → new App → view

If you know Redux, Msg is an action, App is the store state, update is the reducer (with async effects inlined via `block_on`).

32. Walkthrough: reading filter_worklogs without fear

Open these files in order and jot one sentence each:

1. `tui/src/search_dsl.rs` — turns a string into `FilterWorklogsCommand`.
2. `sdk/src/client.rs (filter_worklogs)` — POSTs JSON to `/worklogs/filter`.
3. `api/.../worklog_filter.rs` (controller) — JSON → command → use case → JSON page.
4. `use_cases/.../filter_worklogs.rs` — validates, maps to criteria, calls repo.
5. `core/.../worklog_filter_criteria.rs` — typed filter object.
6. `infrastructure/.../worklog_repository.rs` — SQL + binds + map rows to entities.
7. **use_cases response DTO** — page + statistics for clients.

While reading, notice what each layer is **not** allowed to know. That negative space is the architecture.

33. Performance and simplicity notes (practical)

Worklogger prioritizes clarity over micro-optimization, but a few choices are deliberate:

Choice	Rationale
Pool <code>max_connections(5)</code>	Small service; avoid stampeding Postgres locally
Page size defaults (20/30)	Keep TUI snappy
Export cap 100k rows	Protect memory/time
GIN index on tags	Tag filters use array overlap
Index on (user_id, datetime)	Per-user timeline queries
Soft delete flag vs separate table	Simpler queries with <code>deleted_at IS NULL</code>

When cloning, measure before adding caches. The bottleneck is usually unclear requirements, not missing Redis.

34. Accessibility of the TUI design for keyboard users

The product assumes keyboard-only operation:

- No mouse dependency for core flows.

- Help bar documents shortcuts in-context.
- Modes prevent accidental deletes (confirm dialog).
- Status toasts give non-modal feedback after export/errors.

When extending the TUI, keep destructive actions behind confirm modes and always show the current mode in the title bar (already implemented via Mode → view name).

35. Copy-paste starter: smallest hexagonal Rust service

If you want a **tiny** training repo before cloning all of Worklogger, build this 4-crate skeleton first:

```
trainer/
  Cargo.toml (workspace)
  domain/      # struct Item; trait ItemRepo; DomainError
  app/         # CreateItem use case
  adapter_mem/# InMemoryItemRepo
  adapter_http/# axum POST /items
```

Rules:

1. domain has zero deps beyond thiserror / uuid.
2. app depends only on domain.
3. adapter_* depend on app + domain.
4. Prove you can swap adapter_mem for a sqlx adapter without changing app.

Then graduate to Worklogger’s full workspace — same rules, more modules.

36. Frequently asked questions

Q: Do I need to learn Rust lifetimes before contributing?

A: Learn ownership/&/&mut first. Explicit 'a annotations rarely appear in Worklogger’s public APIs.

Q: Why is the package named core but imported as domain?

A: Cargo allows renaming path dependencies for clearer call sites: domain = { path = "../core", package = "core" }.

Q: Can the TUI work offline against Postgres?

A: Not as designed — it requires the API. You *could* add a direct mode, but that would violate the current adapter boundary on purpose.

Q: Where do I put a new third-party crate?

A: In the outermost crate that needs it. Never add Axum to core “just for a type.”

Q: How do I debug a sqlx query?

A: Run the SQL in `psql` with literal binds, then mirror binds in Rust. Check offline cache after edits.

Q: What Rust edition is this?

A: Edition 2024 across packages (see each `Cargo.toml`). If your toolchain is older, install a newer stable via `rustup`.

Q: Is async mandatory?

A: For this stack yes (`sqlx` + `axum` + `reqwest`). The domain itself could be sync; traits are `async` because adapters are.

37. Study plan: 10 evenings to literacy

Evening	Focus	Concrete task
1	Toolchain	Install Rust; <code>cargo run -p api</code> with Docker/Postgres
2	Ownership	Re-read §4; explain <code>save(&Worklog)</code> aloud
3	Domain	Trace <code>Worklog::soft_delete</code> ; write a new unit test
4	Use cases	Trace <code>create</code> + <code>login</code> ; draw sequence diagram
5	<code>sqlx</code>	Read migration + <code>get/save</code> implementations
6	<code>Axum</code>	Add a trivial <code>/version</code> route returning <code>crate version</code>
7	Auth	Mint token via <code>admin</code> ; call API with <code>curl</code>
8	SDK	Write a 20-line binary that lists worklogs via SDK
9	TUI	Change a theme color; add a key-binding that sets a toast
10	Architecture	Implement Exercise B (in-memory repo) without looking at <code>infra</code>

After evening 10 you should be able to duplicate the project's shape with confidence.

38. Document map (where things live)

Need	Look here
Business rules	core/
Validation messages for API/TUI	use_cases/.../error.rs, command validate
SQL	infrastructure/migrations, *_repository.rs
HTTP routes	api/src/routes/
Auth header parsing	api/src/middleware/auth.rs
JWT crypto	use_cases/src/auth/
Excel layout	use_cases/src/export/
Terminal widgets	tui/src/{ui,components,dialogs}.rs
Search syntax	tui/src/search_dsl.rs
HTTP client	sdk/src/client.rs
User provisioning	admin/src/main.rs
Human product docs	root README.md
This teaching guide	docs/TUTORIAL.md

Extended teaching material §31–§38. The full document (§1–§38 + appendices) is the canonical onboarding tutorial for rebuilding Worklogger as a non-Rust developer.